

Allan Deviation Measurement Procedure

Donni Sow

This document is intended as a manual to measure the Allan deviation of a DDS card. For the `python` code and sample data, click [here](#).

1 Data Recording Procedure

1.1 Apparatus

The pieces of equipment needed for this experiment are as follows:

1. A DDS card and a reference clock.
2. A PicoScope with high sample rate capability (1 GS/s or more)
3. BNC Cables

1.2 Experiment Setup

The setup of the experiment is shown in Figure 1.

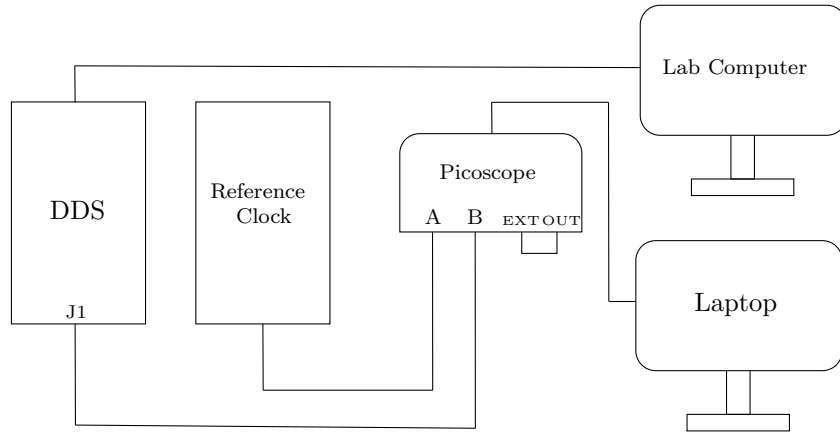


Figure 1: Allan Deviation Measurement Setup. Lab Computer is used to program the DDS, while the Laptop (or any other computer that has PicoScope 7 installed) is used to control and capture the data from the PicoScope.

1.3 Data Acquisition

Follow these steps (the reader can also modify the parameters according to the situation):

1. Determine how long the DDS will run. For short-time averaging, ~ 10 seconds would suffice; for long averaging time, ~ 50 seconds would suffice.
2. Set the PicoScope parameters, like the vertical and time division, and the sample rate. Ideally, the vertical must contain the full sinusoidal wave of the DDS and the reference clock, the horizontal division can contain up to 50 - 500 cycles of the wave and the sample rate has to be at least 4 to 5 times the frequency of the DDS and the reference.
3. Here we can set the triggering rate by the PicoScope's own function generator. Set the PicoScope function generator to square wave, and the frequency to ~ 10 kHz for short averaging time and ~ 100 Hz for long averaging time. Set the PicoScope trigger source to "Ext".
4. Trigger the PicoScope just before the DDS starts running. After the DDS is turned off, stop the triggering.
5. Click the "Save" tab. Choose the waveforms that only contain the full sinusoidal waves present for both DDS and the reference. Save it as a Tab-Delimited text document.

2 Data Analysis Procedure

In this section, we will describe the standard way of processing the data.

2.1 Data Fitting and Cleaning

The Tab-Delimited text document that we have is in its raw form. To turn the data into phase difference data, one can use the `python` script in the link given [here](#). Use the function `f` in the script to process the raw data. The example is given below.

```
1 # Let's say the raw data is named "data" and the script is in the same folder as
  the raw data. Then, to process the data, one can do
2 phase_difference = f("data", 80e6)
3 # One can also replace "data" with the path of the data folder, if the data
  folder is not in the same folder as the python script.
```

The variable `phase_difference` is a pandas series containing the phase difference in units of seconds for each file in the folder. See section [3.1](#) for more details on the functions.

2.2 Analyzing the Data

Once we get the phase difference after the cleaning and preprocessing, follow these steps:

1. Convert phase difference to fractional frequency using the function `allantools.phase2frequency`. Refer to this for the documentation of `allantools`[\[2\]](#)
2. Plot fractional frequency vs index number (or alternatively, fractional frequency vs time).
3. Look for an obvious outlier. The first few are usually outliers. If present, remove them.
4. Compute the OADEV using the `allantools` library. Be sure to input the correct data type and triggering rate.

Additionally, one can remove the drift by calculating the overlapping Hadamard deviation¹ (using the same library) instead, or by fitting the fractional frequency (usually a linear or polynomial fit) after step 3 to correct for the frequency offset. The former method is easier and preferable. A random walk ($\propto \tau^{1/2}$) and random run ($\propto \tau^{3/2}$) might present on the Hadamard/drift-corrected Allan deviation. Refer to [1] for details on frequency stability analysis. The matter of plotting the data is up to the reader.

3 Appendices

3.1 The Code

1. `cleaning` is a function that pre-processes the data. The raw data from the picoscope needs to be cleaned and pre-processed to convert the sinusoidal data points into phase difference data. One can easily do this using the function `cleaning`. The input to this function is the raw data, and it outputs two cleaned versions of the data, where the time columns is converted to seconds.
2. `model`: It is a function that describes a general sine wave curve
3. `split_n_fit`: Takes a data frame as an input and output the fit parameter, calculated using `scipy`. The parameters also have been corrected for negative amplitude.
4. `read_folder`: Read every single file in a given folder as a data frame. The input is the directory of the folder. It returns a collection of data frames.
5. `summarize`: Summarize the frequency and phase data of each data frame²
6. `f`: It is a function that combines all previous functions. The input is the folder directory and a rough frequency guess. Keep in mind that the reference and the DDS frequency have to be approximately the same; otherwise, the fit won't give a reliable result, or the fitting fails. It outputs a data frame of phase difference in units of seconds.

References

- [1] W J Riley. *Handbook of frequency stability analysis*. en. Tech. rep. Gaithersburg, MD: National Institute of Standards and Technology, 2008. DOI: [10.6028/nist.sp.1065](https://doi.org/10.6028/nist.sp.1065). URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication1065.pdf> (visited on 07/17/2025).
- [2] Anders E. E. Wallin et al. “allantools: Allan deviation calculation”. In: *Astrophysics Source Code Library* (Apr. 2018). ADS Bibcode: 2018ascl.soft04021W, ascl:1804.021. URL: <https://ui.adsabs.harvard.edu/abs/2018ascl.soft04021W> (visited on 07/18/2025).

¹Hadamard deviation is gives very similar results to a drift-free Allan deviation. See the NIST Handbook.

²This function is obsolete, because each data frame has only one phase and one frequency value. But it does work, it is just an inconvenient way to achieve our purpose.